

⚡ 08. The Problem with Async Errors in Express

💣 Why Async Errors Break Express 4

Synchronous errors are automatically handled by Express. However, **unhandled rejected promises in async route handlers are NOT caught automatically in Express 4!** The request either hangs indefinitely or crashes the process.

😓 The Ugly Solution: Repetitive Try-Catch

Manually wrapping every single async route in try-catch produces bloated, unmaintainable boilerplate:

Ugly Try/Catch Pattern

```
app.get("/chats/:id", async (req, res, next) => {
  try {
    let chat = await Chat.findById(req.params.id);
    if (!chat) throw new ExpressError(404, "Chat Not Found!");
    res.render("show.ejs", { chat });
  } catch (err) {
    next(err); // Repetitive catch block in every single route!
  }
});
```



BOILERPLATE OVERLOAD: Real projects contain dozens of routes. Writing try-catch blocks everywhere severely damages readability and maintainability!

🌟 The Pro Solution: wrapAsync & Mongoose Errors

🌟 The wrapAsync Higher-Order Function

`wrapAsync` is a higher-order function that wraps async handlers, catching any rejected promise and piping it directly to `.catch(next)`:

```
utils/wrapAsync.js - Magic Wrapper

function wrapAsync(fn) {
  return function (req, res, next) {
    fn(req, res, next).catch(next); // 🐞 Automatically passes rejected promise to next(err)
  };
}
module.exports = wrapAsync;
```

🌟 Clean Route Implementation

```
index.js - Clean Async Route

// Zero try-catch boilerplate! Clean, expressive code:
app.get("/chats/:id", wrapAsync(async (req, res) => {
  let chat = await Chat.findById(req.params.id);
  if (!chat) throw new ExpressError(404, "Chat Not Found!");
  res.render("show.ejs", { chat });
})));
```

🎯 Handling Specific Mongoose Errors

Mongoose Error	Trigger Cause	Handling Strategy
ValidationError	Schema validation fails (e.g. required field missing)	Extract <code>err.errors</code> and respond with 400 Bad Request.
CastError	Malformed ObjectId passed in URL (e.g. /chats/123)	Check <code>err.name === "CastError"</code> and return 400 "Invalid ID format".
MongoServerError (11000)	Duplicate unique index (e.g. email already exists)	Detect code 11000 and return "Email already registered".